

SYSTEM REQUIREMENTS – KeyShare



1. Chosen Development Model

Development Model: Agile (Scrum)

Reasons:

The Agile development model was chosen because it allows flexibility and continuous improvement throughout the project. Since KeyShare includes multiple features such as booking, listings, and user interaction, Agile enables the team to:

- Adapt to changes easily
- Develop features incrementally
- Test and improve continuously
- Deliver a working prototype faster

2. User Requirements

a. Stakeholders

1. Renters (End Users)

- Individuals who want to rent vehicles
- Interested in ease of use, quick booking, and affordable options

2. Vehicle Owners

- Individuals who want to rent out their cars
- Interested in earning money and managing listings easily

3. Businesses / Companies

- Companies renting fleets of vehicles

- Interested in managing multiple cars and maximizing usage

4. Admin

- Manages the platform
- Ensures system security, monitors users, and resolves issues

5. Developers

- Build and maintain the system
- Ensure performance, updates, and bug fixes

b. User Stories

1. As a **user**, I want to register an account so that I can access the platform.
2. As a **user**, I want to log in so that I can manage my bookings and listings.
3. As a **vehicle owner**, I want to list my car so that I can earn money.
4. As a **vehicle owner**, I want to edit or delete my listings so that I can keep them updated.
5. As a **renter**, I want to search for cars so that I can find suitable options.
6. As a **renter**, I want to filter cars by price and location so that I can find what fits my needs.
7. As a **renter**, I want to view detailed car information so that I can make informed decisions.
8. As a **renter**, I want to book a car so that I can use it when needed.
9. As a **vehicle owner**, I want to accept or reject booking requests so that I can control availability.
10. As a **user**, I want to leave reviews so that I can share my experience.
11. As a **user**, I want to view ratings so that I can choose reliable cars/users.
12. As a **user**, I want to receive notifications so that I stay updated on bookings.

3. Functional Requirements

a. Description

1. The system should allow users to register accounts.
2. The system should allow users to log in and log out.
3. The system should allow users to update their profile.
4. The system should allow vehicle owners to add car listings.
5. The system should allow owners to edit listings.
6. The system should allow owners to delete listings.
7. The system should display available cars.

8. The system should allow users to search cars.
9. The system should allow filtering by price, location, and type.
10. The system should display detailed vehicle information.
11. The system should allow users to upload car images.
12. The system should allow booking requests.
13. The system should allow owners to accept/reject bookings.
14. The system should track booking status.
15. The system should prevent double booking.
16. The system should allow users to leave reviews.
17. The system should display ratings.
18. The system should send notifications for bookings.
19. The system should include a messaging/chat feature.
20. The system should allow admins to manage users.
21. The system should allow admins to remove inappropriate listings.
22. The system should display an availability calendar.
23. The system should allow users to cancel bookings.
24. The system should simulate payments (optional).
25. The system should allow multi-vehicle management for businesses.

b. Acceptance Criteria

1. User Registration

- User enters valid email and password
- System validates input
- Account is created successfully
- Confirmation message is displayed

2. User Login

- User enters correct credentials
- System verifies credentials
- User is redirected to dashboard
- Error message shown for invalid login

3. Add Vehicle Listing

- Owner enters vehicle details (model, price, location)
- Owner uploads images
- System saves listing
- Listing appears in search results

4. Booking System

- User selects a vehicle
- User selects available dates
- Booking request is sent
- Owner receives notification
- Booking status updates after approval/rejection

5. Search and Filter

- User enters search criteria
- System displays matching vehicles
- Filters refine results instantly
- No results message shown if none available

4. Non-Functional Requirements

a. Description

1. The system should load quickly.
2. The system should be user-friendly.
3. The system should be responsive (mobile-friendly).
4. The system should be secure.
5. The system should handle multiple users simultaneously.
6. The system should ensure data privacy.
7. The system should be reliable with minimal downtime.
8. The system should scale as users increase.
9. The system should provide fast search results.
10. The system should have clear navigation.
11. The system should provide error messages.
12. The system should support different browsers.
13. The system should maintain data consistency.
14. The system should back up data regularly.
15. The system should encrypt sensitive data.
16. The system should have role-based access control.
17. The system should respond quickly to user actions.
18. The system should have minimal bugs.
19. The system should be easy to maintain.
20. The system should log user activity.
21. The system should ensure high availability.
22. The system should provide notifications in real time.

b. Acceptance Criteria

1. Performance

- Pages load in under 2 seconds
- Search results appear in under 1 second
- Booking process completes in under 5 seconds

2. Usability

- Users can navigate without training
- Interface is clear and simple
- Key actions are accessible within 3 clicks

3. Security

- Passwords are encrypted
- Users must log in to access features
- Unauthorized access is blocked

4. Reliability

- System uptime is at least 99%
- No data loss during normal operation
- Errors are handled gracefully

5. Compatibility

- Works on Chrome, Firefox, Edge
- Works on desktop and mobile devices
- UI adjusts to different screen sizes

5. Application Specifications

a. Architecture

The system follows a **three-tier architecture**:

- **Frontend:** User interface (web/mobile app)
- **Backend:** Handles logic (authentication, bookings, listings)
- **Database:** Stores users, vehicles, bookings

The frontend sends requests to the backend, which processes data and interacts with the database.

b. Database Model

Main Tables:

- **Users** (id, name, email, password, role)
- **Vehicles** (id, owner_id, model, price, location, availability)
- **Bookings** (id, user_id, vehicle_id, dates, status)
- **Reviews** (id, user_id, vehicle_id, rating, comment)

Relationships:

- One user → many vehicles
- One vehicle → many bookings
- One booking → one user & one vehicle

Constraints:

- Email must be unique
- Bookings must reference valid users and vehicles
- No overlapping bookings allowed

c. Technologies Used

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js / Python
- **Database:** MySQL

Reason: These technologies are widely used, easy to learn, and suitable for building scalable web applications.

d. User Interface Design

The application includes:

- Login/Register page
- Dashboard
- Vehicle listing page
- Search & filter page
- Vehicle details page
- Booking page

Design Features:

- Clean layout
- Easy navigation
- Clear buttons (Book, Search, Add Vehicle)
- Mobile-friendly interface

e. Security Measures

- User authentication (login system)
- Password encryption (hashed passwords)
- Role-based access control (admin vs user)
- Secure data storage
- Input validation to prevent attacks
- Session management for logged-in users